

# LAB REPORT 06

## THE FOR LOOP IN C++

### 1. OBJECTIVE

- To understand the syntax and flow control of a for loop.
- To implement simple counter-controlled loops.
- To implement and trace nested for loops.

### 2. THEORETICAL BACKGROUND

**The for Loop.** A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times. The for loop is used when a value runs from a starting point to an ending point with a constant increment or decrement.

**Flow control in a for loop.** The initialization of variable is carried out first, and only once. This step allows you to declare and initialize any loop control variables. Next, the condition is evaluated. If it is true, the body of the loop is executed. After the body of the for loop executes, the flow of control jumps back up to update the variable. The condition is now evaluated again. If it is true, the loop executes and the process repeats itself. After the condition becomes false, the for loop terminates.

**Nested for Loop.** A nested for loop is simply a for loop inside another for loop. Nested loops are often used for printing grid-like patterns or iterating through multi-dimensional data structures.

### 3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

### 4. EXPERIMENTAL TASKS

#### Task 1: for Loop

Write a C++ program that uses a for loop to display numbers from 1 to 10. Display each number on a separate line.

```
#include <iostream>
using namespace std;

int main() {
    // A simple for loop to display numbers from 1 to 10
    for (int i = 1; i <= 10; i++) {
        cout << i << endl;
    }
    return 0;
}
```

#### Task 2: Nested for Loop: Multiplication Table

Write a C++ program that generates a multiplication table from 1 to 10. Use nested for loops to iterate through the rows and columns of the table. Display the multiplication table in a formatted manner.

```
#include <iostream>
using namespace std;

int main() {
    // Nested loop to generate a 10x10 multiplication table
    for (int row = 1; row <= 10; row++) {
        for (int col = 1; col <= 10; col++) {
            cout << (row * col) << "\t"; // Use tab for formatting
        }
        cout << "\n"; // New line at the end of each row
    }
    return 0;
}
```

### Task 3: Sum using for Loop

Write a C++ program that prompts the user to enter a positive integer. Use a for loop to calculate the sum of all numbers from 1 to the entered integer. Display the calculated sum.

```
#include <iostream>
using namespace std;

int main() {
    int limit;
    int sum = 0;

    // Prompt user for input
    cout << "Enter a positive integer: ";
    cin >> limit;

    // Calculate sum using a for loop
    for (int i = 1; i <= limit; i++) {
        sum += i;
    }

    cout << "The sum of numbers from 1 to " << limit << " is: " << sum <<
endl;
    return 0;
}
```

### Task 4: Nested Pattern Printing

Write a C++ program that uses nested for loops to print a pattern of asterisks. The pattern should form a triangle shape, with each row having an increasing number of asterisks. The number of rows in the pattern should be determined by user input.

```
#include <iostream>
using namespace std;

int main() {
    int rows;

    cout << "Enter the number of rows for the triangle: ";
    cin >> rows;

    // Outer loop for rows
    for (int i = 1; i <= rows; i++) {
        // Inner loop for asterisks in each row
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << "\n"; // Move to the next line
    }
    return 0;
}
```

## 5. CONCLUSION

This lab provided practical experience with the for loop, a fundamental repetition structure in C++. By examining the initialization, condition, and update phases of the loop, students learned how to control iteration flow accurately. Furthermore, the introduction of nested for loops demonstrated how to handle multi-dimensional iteration, enabling the creation of complex outputs such as formatted multiplication tables and geometric patterns. These skills are essential for implementing algorithms that require repetitive processing.